

Definición y ejecución de tests unitarios con Pytest



Eva Pérez Carmona
2ºDAM
12/02/2026

ÍNDICE

Definición y ejecución de tests unitarios con Pytest	1
Desarrollo	3
- TEST RED	3
- TEST GREEN	3
- TEST REFACTOR	4
Webgrafía	6

Desarrollo

- TEST RED

Para este test hemos dejado la clase factorial.py vacía y con ello, sea el test que sea, va a dar error al no haber contenido en la clase principal.

```

1 import factorial
2 import pytest
3
4 def test_factorial_uno():
5     assert factorial(1) == 1
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

```

```

test_factorial_uno
def test_factorial_uno():
    assert factorial(1) == 1
    .....
E       TypeError: 'module' object is not callable
test_factorial.py:5: TypeError
===== short test summary info =====
FAILED test_factorial.py::test_factorial_uno - TypeError: 'module' object is ...
===== 1 failed in 0.04s =====
Finished running tests!

```

- TEST GREEN

Para este apartado, hemos utilizado el mínimo código posible para que el test pase. Contando con las condiciones del enunciado, hemos comprobado que al pasarle 5 a factorial, el resultado sea 120.

```

1 from factorial import factorial
2 import pytest
3
4 def test_factorial_uno():
5     assert factorial(1) == 1
6
7 def test_factorial_cinco():
8     assert factorial(5) == 120
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

```

```

Running pytest with args: ['-p', 'vscode.pytest', '--rootdir=c:\\Users\\alumno\\Desktop\\segundo_DAM\\des_inter\\tema5\\tdd_factorial_Pérez_Eva', '-c:c:\\Users\\alumno\\Desktop\\segundo_DAM\\des_inter\\tema5\\tdd_factorial_Pérez_Eva\\test_factorial.py::test_factorial_cinco']
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: c:\\Users\\alumno\\Desktop\\segundo_DAM\\des_inter\\tema5\\tdd_factorial_Pérez_Eva
collected 1 item

test_factorial.py . [100%]

===== 1 passed in 0.02s =====
Finished running tests!

```

- *TEST REFACTOR*

Para este apartado, he capturado las excepciones de manera que sólo dé error el primer test. Para seguir controlando esto, hemos añadido líneas de código para que Python entienda que nosotros ya sabemos lo que va a salir, de esta manera, todos los test acaban pasando (última captura).

The screenshot shows the Visual Studio Code editor with a file named `test_factorial.py` open. The file contains the following code:

```
1 from factorial import factorial
2 import pytest
3
4 def test_factorial_uno():
5     assert factorial(1) == 1
6
7 def test_factorial_cinco():
8     assert factorial(5) == 120
9
10 def test_factorial_negativo():
11     with pytest.raises(ValueError):
12         factorial(-1)
13
14 def test_factorial_no_entero():
15     with pytest.raises(TypeError):
16         factorial(5.5)
```

The bottom panel shows the output of running the tests. The terminal displays the following text:

```
Running pytest with args: ['-p', 'vscode_pytest', '--rootdir=c:\\Users\\alumno\\Desktop\\segundo DAM\\des_inter\\temas\\tdd_factorial Pérez Eva', 'c:\\Users\\alumno\\Desktop\\segundo DAM\\des_inter\\temas\\tdd_factorial Pérez Eva\\test_factorial.py::test_factorial_negativo']
----- test session starts -----
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: c:\\Users\\alumno\\Desktop\\segundo DAM\\des_inter\\temas\\tdd_factorial Pérez Eva
collected 1 item

test_factorial.py . [100%]

----- 1 passed in 0.01s -----
Finished running tests!
```

The right sidebar shows the test results for the workspace, indicating that the test `test_factorial_negativo` passed.

The screenshot shows the Visual Studio Code editor with a file named `test_factorial.py` open. The file contains the following code:

```
1 from factorial import factorial
2 import pytest
3
4 def test_factorial_uno():
5     assert factorial(1) == 1
6
7 def test_factorial_cinco():
8     assert factorial(5) == 120
9
10 def test_factorial_negativo():
11     with pytest.raises(ValueError):
12         factorial(-1)
13
14 def test_factorial_no_entero():
15     with pytest.raises(TypeError):
16         factorial(5.5)
```

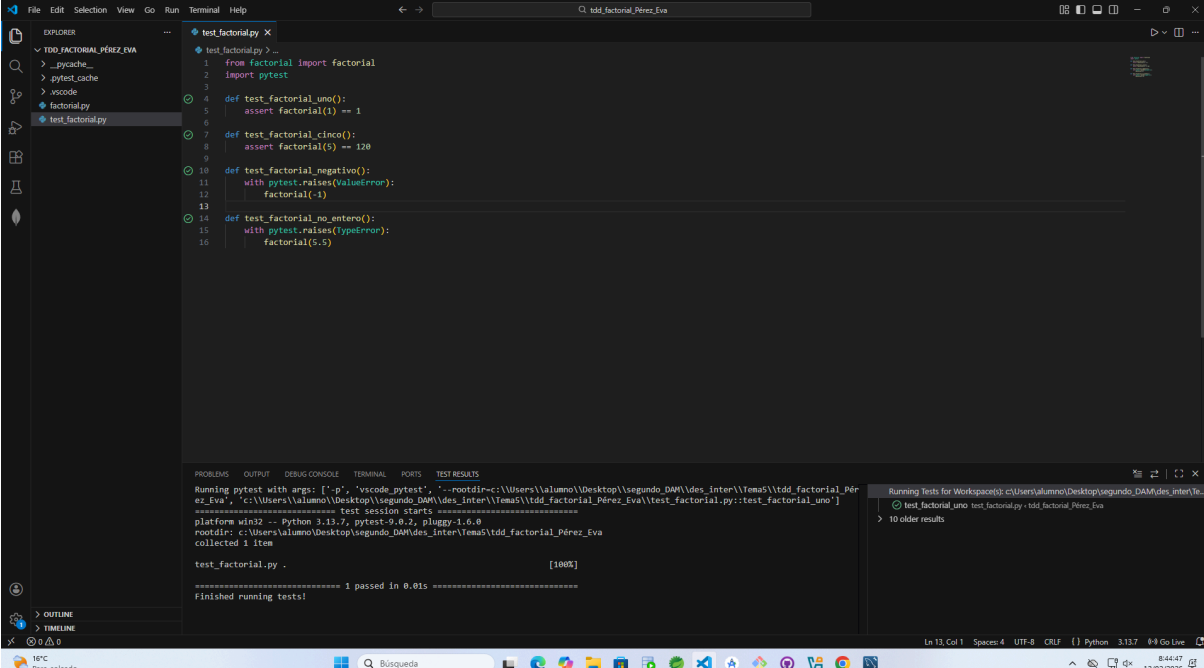
The bottom panel shows the output of running the tests. The terminal displays the following text:

```
Running pytest with args: ['-p', 'vscode_pytest', '--rootdir=c:\\Users\\alumno\\Desktop\\segundo DAM\\des_inter\\temas\\tdd_factorial Pérez Eva', 'c:\\Users\\alumno\\Desktop\\segundo DAM\\des_inter\\temas\\tdd_factorial Pérez Eva\\test_factorial.py::test_factorial_no_entero']
----- test session starts -----
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: c:\\Users\\alumno\\Desktop\\segundo DAM\\des_inter\\temas\\tdd_factorial Pérez Eva
collected 1 item

test_factorial.py . [100%]

----- 1 passed in 0.01s -----
Finished running tests!
```

The right sidebar shows the test results for the workspace, indicating that the test `test_factorial_no_entero` passed.



The screenshot shows a Visual Studio Code editor window with a Python file named `test_factorial.py` open. The file contains several test functions for a factorial function using `pytest`. The tests are:

- `test_factorial_uno()`: Asserts `factorial(1) == 1`.
- `test_factorial_cinco()`: Asserts `factorial(5) == 120`.
- `test_factorial_negativo()`: Asserts that `factorial(-1)` raises a `ValueError`.
- `test_factorial_no_entero()`: Asserts that `factorial(5.5)` raises a `TypeError`.

The bottom panel of the editor shows the output of running the tests. It indicates that all tests passed successfully.

```
Running pytest with args: ['-p', 'vscode.pytest', '--rootdir=c:\\Users\\alumno\\Desktop\\segundo DAM\\des_inter\\Temas\\tdd_factorial_Pérez_Eva', 'c:\\Users\\alumno\\Desktop\\segundo DAM\\des_inter\\Temas\\tdd_factorial_Pérez_Eva\\test_factorial_uno']
===== Test session starts =====
platform win32 -- Python 3.11.7, pytest-8.0.2, pluggy-1.6.0
rootdir: c:\\Users\\alumno\\Desktop\\segundo DAM\\des_inter\\Temas\\tdd_factorial_Pérez_Eva
collected 1 item

test_factorial.py . [100%]

===== 1 passed in 0.01s =====
Finished running tests!
```

Webgrafía

- <https://experts-deny-b9a.craft.me/y3eKOUPLrtEM6g/b/1D6BDE8D-54BC-4B30-9559-CF383846E398/Ejercicio-Resuelto-Aplicando-TDD>
- <https://experts-deny-b9a.craft.me/GYhq8YX7HUNPsL>
- https://iabigdata-soka-4ae9e223e32444ac5ae3d78afbd55fd9aa6dalcl9d9679bf.gitlab.io/post/2024-06-03-pia_pytest_exceptions/